

UNIVERSIDADE ESTADUAL DO NORTE FLUMINENSE
LABORATÓRIO DE ENGENHARIA E EXPLORAÇÃO DE
PETRÓLEO

PROJETO DE ENGENHARIA
DESENVOLVIMENTO DO SOFTWARE
FERRAMENTA PARA GERAÇÃO DA CURVA DE FLUXO
FRACIONÁRIO
TRABALHO DA DISCIPLINA PROGRAMAÇÃO PRÁTICA

Versão 1:
Fabiane da Silva Barros
Prof. André Duarte Bueno

MACAÉ - RJ
Dezembro - 2025

Sumário

1	Concepção	6
1.1	Nome do sistema/produto	6
1.2	Especificação	6
1.3	Requisitos	7
1.3.1	Requisitos funcionais	7
1.3.2	Requisitos não funcionais	8
1.4	Casos de uso	8
1.4.1	Diagrama de caso de uso geral	8
1.4.2	Diagrama de caso de uso específico	10
2	Elaboração	11
2.1	Análise de domínio	11
2.2	Formulação teórica	12
2.2.1	Implementação do Modelo Tabular (Interpolação Linear)	13
2.2.2	Implementação do Modelo Analítico (Correlações de Corey)	14
2.3	Identificação de pacotes – assuntos	15
2.4	Diagrama de pacotes – assuntos	16
3	AOO – Análise Orientada a Objeto	17
3.1	Diagramas de classes	17
3.1.1	Dicionário de classes	19
3.2	Diagrama de sequência – eventos e mensagens	19
3.2.1	Diagrama de sequência geral	19
3.2.2	Diagrama de sequência específico	21
3.3	Diagrama de comunicação – colaboração	21
3.4	Diagrama de estado	22
3.5	Diagrama de atividades	23
4	Projeto	25
4.1	Projeto do sistema	25
4.2	Projeto orientado a objeto – POO	26
4.2.1	Efeitos do projeto no modelo estrutural:	26
4.2.2	Efeitos do projeto no modelo dinâmico:	26
4.2.3	Efeitos do projeto nos atributos:	26

4.2.4	Efeitos do projeto nos métodos:	27
4.2.5	Efeitos do projeto nas heranças:	27
4.2.6	Efeitos do projeto nas associações:	27
4.2.7	Efeitos do projeto nas otimizações:	27
4.3	Diagrama de componentes	27
4.4	Diagrama de implantação	28
4.4.1	Lista de características <<features>>	29
4.4.2	Tabela classificação sistema	30
5	Teste	32
5.1	Teste 1: Validando o Ciclo 1 - Modelo Tabelado	32
5.1.1	Arquivo de entrada de dados	32
5.1.2	Arquivo de saída de dados (resultados da tela)	33
5.1.3	Figuras	33
5.2	Teste 2: Validando o Ciclo 2 - Modelo Corey	34
5.2.1	Arquivo de entrada de dados	35
5.2.2	Arquivo de saída de dados (resultados da tela)	35
5.2.3	Figuras	35
5.3	Teste 3: Validação com Literatura (Tarek Ahmed)	36
5.3.1	Arquivo de entrada de dados	36
5.4	Teste 4: Validação da Camada de Apresentação (Gnuplot)	39
5.4.1	Arquivo de entrada de dados	40
5.4.2	Figuras	40
A	Formato do Arquivo de Entrada	42
A.1	Exemplo de Arquivo de Entrada (Modelo Tabelado)	42
A.2	Exemplo de Arquivo de Entrada (Modelo Corey)	43
B	Guia de Compilação e Execução	44
B.1	Pré-requisitos	44
B.2	Compilação do Projeto	44
B.3	Execução e Teste	45

Lista de Figuras

1.1	Diagrama de caso de uso – Caso de uso geral	9
1.2	Diagrama de caso de uso específico – Título.	10
2.1	Curva de fluxo fracionário (f_w) característica, mostrando a sua forma de "S". Fonte: Adaptado de Ahmed, 2019 [1].. . . .	13
2.2	Diagrama de Pacotes.	16
3.1	Diagrama de classes.	18
3.2	Diagrama de sequência.	20
3.3	Diagrama de sequência.	21
3.4	Diagrama de comunicação.	22
3.5	Diagrama de máquina de estado.	23
3.6	Diagrama de atividades.	24
4.1	Diagrama de componentes.	28
4.2	Diagrama de implantação.	29
5.1	Tela do programa mostrando terminal executando.	34
5.2	Tela do programa mostrando gráfico (<i>Excel</i>) gerado a partir do <code>output.csv</code>	34
5.3	Tela do programa mostrando curva do modelo de Corey.	36
5.4	Tela do programa mostrando curva do Exemplo 14-5 do livro <i>Reservoir Engineering Handbook</i> de Tarek Ahmed [1] (com $\mu_o =$ 1.0).	38
5.5	Curva do Exemplo 14-5 do livro <i>Reservoir Engineering Handbook</i> de Tarek Ahmed [1] (com $\mu_o = 0.5, \mu_o = 1.0, \mu_o = 5$ e $\mu_o = 10$). . .	39
5.6	Tela do programa mostrando captura de tela a janela do Gnuplot. . .	40

Listagens

5.1	Teste-01.in.	32
5.2	Teste-02.in.	35
5.3	Teste-Tarek-1cp.in.	37
A.1	Exemplo de arquivo de entrada para modelo tabelado.	42
A.2	Exemplo de arquivo de entrada para modelo Corey.	43
B.1	Comando para navegar até a pasta do projeto.	44
B.2	Limpando arquivos antigos.	44
B.3	Comando de compilação.	45
B.4	Executando o Teste 1.	45
B.5	Executando a Validação com Literatura.	45

Capítulo 1

Concepção

Apresenta-se neste capítulo do projeto de engenharia a concepção, a especificação do sistema a ser modelado e desenvolvido.

1.1 Nome do sistema/produto

Abaixo a Tabela 1.1 com o nome do Sistema/Produto: Ferramenta para Geração da Curva de Fluxo Fracionário.

Tabela 1.1: Nome do Sistema/Produto.

Nome	Ferramenta para Geração da Curva de Fluxo Fracionário
Componentes principais	1. Módulo de Entrada (Leitor de arquivo de texto) 2. Módulo de Dados (Modelos de Permeabilidade) 3. Módulo de Cálculo (Equação de Buckley-Leverett) 4. Módulo de Saída (Gerador de .csv e interface com Gnuplot)
Missão	Missão, Calcular e visualizar a curva de fluxo fracionário (f_w vs. S_w) para auxiliar na análise de deslocamento bifásico em meios porosos.

1.2 Especificação

A especificação desta solução de engenharia define uma ferramenta computacional analítica, de caráter didático e profissional, implementada como um executável de linha de comando. O sistema irá abstrair a complexidade do modelo matemático de fluxo fracionário, exigindo do usuário apenas um arquivo de

configuração em texto simples. Este arquivo deverá conter os parâmetros reológicos dos fluidos (viscosidades) e os dados de permeabilidade relativa, que são suportados em dois formatos: (a) dados tabulares discretos (S_w, k_{rw}, k_{ro}), que serão processados via interpolação linear; ou (b) parâmetros para o modelo analítico de Corey [3]). O software processará esses dados, calculará a curva f_w vs S_w e gerará dois produtos: um arquivo de dados `.csv` para análise externa e uma visualização gráfica imediata, invocando o Gnuplot para plotar a curva "S" característica.

1.3 Requisitos

Apresenta-se nesta seção os requisitos funcionais e não funcionais.

1.3.1 Requisitos funcionais

Apresenta-se a seguir os requisitos funcionais.

RF-01	O sistema deve ler um arquivo de entrada de texto contendo os parâmetros da simulação (viscosidades e dados de permeabilidade).
RF-02	O sistema deve permitir que os dados de permeabilidade relativa sejam fornecidos de duas formas: (a) uma tabela de pontos (S_w, k_{rw}, k_{ro}) ou (b) parâmetros para um modelo analítico (Correlações de Corey[3]).
RF-03	O sistema deve calcular os valores de fluxo fracionário (f_w) para um intervalo de saturação de água (S_w) de 0 a 1.
RF-04	O sistema deve gerar um arquivo de saída em formato de texto (ex: <code>.csv</code>) contendo os pares de dados (S_w, f_w) calculados.
RF-05	O sistema deve invocar um processo externo (Gnuplot) para plotar automaticamente o gráfico de f_w vs. S_w .
RF-06	O sistema deve informar ao usuário em caso de erro na leitura do arquivo de entrada (ex: arquivo não encontrado, formato inválido).

1.3.2 Requisitos não funcionais

RNF-01	Os cálculos de permeabilidade relativa, quando tabulados, devem usar interpolação linear.
RNF-02	O programa deverá ser multi-plataforma, podendo ser executado em <i>Windows</i> e GNU/Linux.
RNF-03	O software deve ser desenvolvido na linguagem C++ utilizando o paradigma da Orientação a Objetos.
RNF-04	O software deve ser executável via terminal (linha de comando).

1.4 Casos de uso

A Tabela 1.2 mostra a descrição de um caso de uso.

Tabela 1.2: Caso de uso 1.	
Nome do caso de uso:	Executar Cálculo e Analisar Resultados
Resumo/descrição:	O usuário executa a ferramenta, que lê os dados de entrada, calcula a curva de fluxo fracionário e exibe o gráfico resultante.
Etapas:	1. Usuário executa o programa via terminal, fornecendo o nome do arquivo de entrada. 2. O sistema lê e valida o arquivo de entrada. 3. O sistema calcula a curva de f_w vs. S_w. 4. O sistema salva os resultados em um arquivo <code>.csv</code>. 5. O sistema invoca o Gnuplot para exibir o gráfico. 6. O usuário analisa o gráfico e fecha a janela.
Cenários alternativos:	2a. O arquivo de entrada não existe ou está mal formatado: O sistema exibe uma mensagem de erro clara e encerra. 5a. O Gnuplot não está instalado ou não foi encontrado no PATH do sistema: O sistema exibe um aviso, mas informa que o arquivo de dados <code>.csv</code> foi gerado com sucesso.

1.4.1 Diagrama de caso de uso geral

O diagrama de caso de uso geral da Figura 1.1 mostra o ator principal, o Usuário (Engenheiro/Estudante), e as interações primárias que ele pode ter com o sistema.

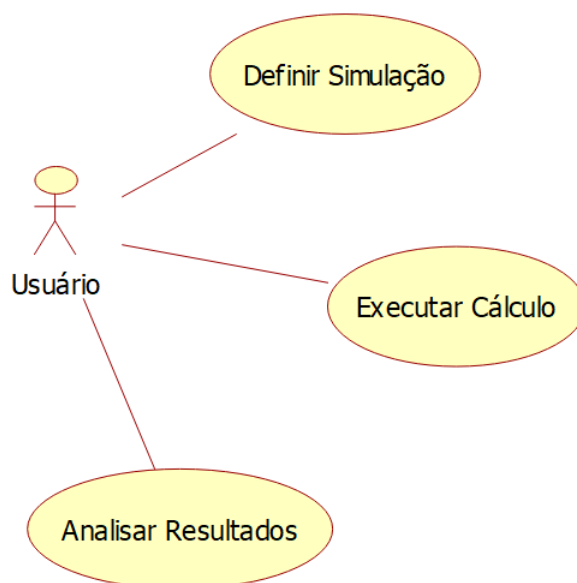


Figura 1.1: Diagrama de caso de uso – Caso de uso geral

1.4.2 Diagrama de caso de uso específico

.O caso de uso Executar Cálculo é detalhado na Figura 3.2.2. Ele é composto por várias sub tarefas que são obrigatoriamente incluídas em sua execução.

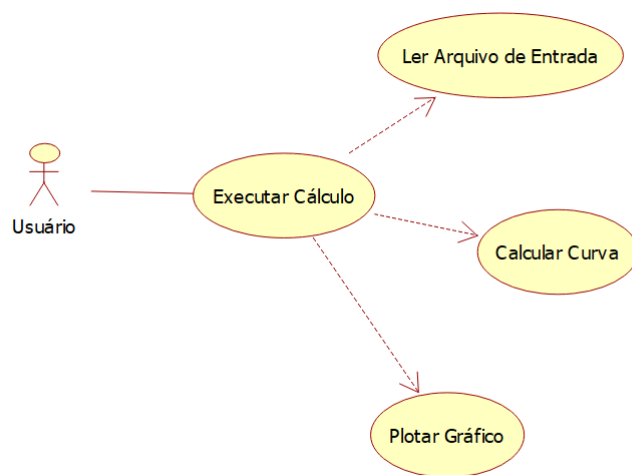


Figura 1.2: Diagrama de caso de uso específico – Título.

Capítulo 2

Elaboração

Este capítulo detalha o domínio do problema e a lógica de engenharia.

2.1 Análise de domínio

O domínio deste projeto situa-se na interseção da Engenharia de Reservatórios com a Modelagem Computacional. A seguir, detalha-se a análise deste domínio e as decisões de arquitetura de software que ela motivou.

- Justificativa da Plataforma Tecnológica: A linguagem C++ (padrão C++ 17) foi selecionada como base tecnológica. A decisão se justifica pelo seu comprovado desempenho computacional, controle de baixo nível de memória e vasta biblioteca padrão (STL). Essa escolha garante que o núcleo de cálculo seja eficiente e, futuramente, possa ser facilmente integrado como uma biblioteca em simuladores de reservatório mais complexos. O desenvolvimento visa a portabilidade entre *Windows* e GNU/Linux, utilizando ferramentas de compilação padrão (g++).
- Arquitetura de Dados: Optou-se por uma arquitetura *serverless* e sem banco de dados. A persistência dos dados de entrada é gerenciada pelo usuário via arquivos de texto `.txt` com formato baseado em palavras-chave. Esta decisão mitiga a complexidade de *deploy* e configuração, alinhando-se à natureza de uma ferramenta de análise rápida. O protocolo de saída (`.csv`) foi escolhido por sua universalidade, permitindo a interoperabilidade com qualquer software de planilha ou *script* de pós-processamento.
- Gerenciamento de Dependências: O projeto terá uma única dependência externa principal: o Gnuplot. Para manter o software leve e evitar a complexidade de linkagem de bibliotecas gráficas, o Gnuplot será invocado como um processo de sistema externo (`system()`). A classe Gnuplot do nosso projeto atuará como uma Facade, encapsulando essa chamada.

- Modelo de Controle: O fluxo de execução do software é puramente sequencial e síncrono (Leitura -> Cálculo -> Plotagem). Dada a baixa complexidade computacional do modelo 1D, não há necessidade de implementação de concorrência ou processamento paralelo, o que simplifica o design.

2.2 Formulação teórica

O núcleo deste projeto é a implementação da equação de fluxo fracionário (f_w) de Buckley-Leverett[2], conforme definido no documento do Desafio Tecnológico :

$$f_w = \frac{1}{1 + \frac{k_{ro}(S_w)}{\mu_o} \cdot \frac{\mu_w}{k_{rw}(S_w)}}$$

A curva, resultado da equação do fluxo fracionário possui uma característica de forma “S”, como está apresentado na Figura 2.1.

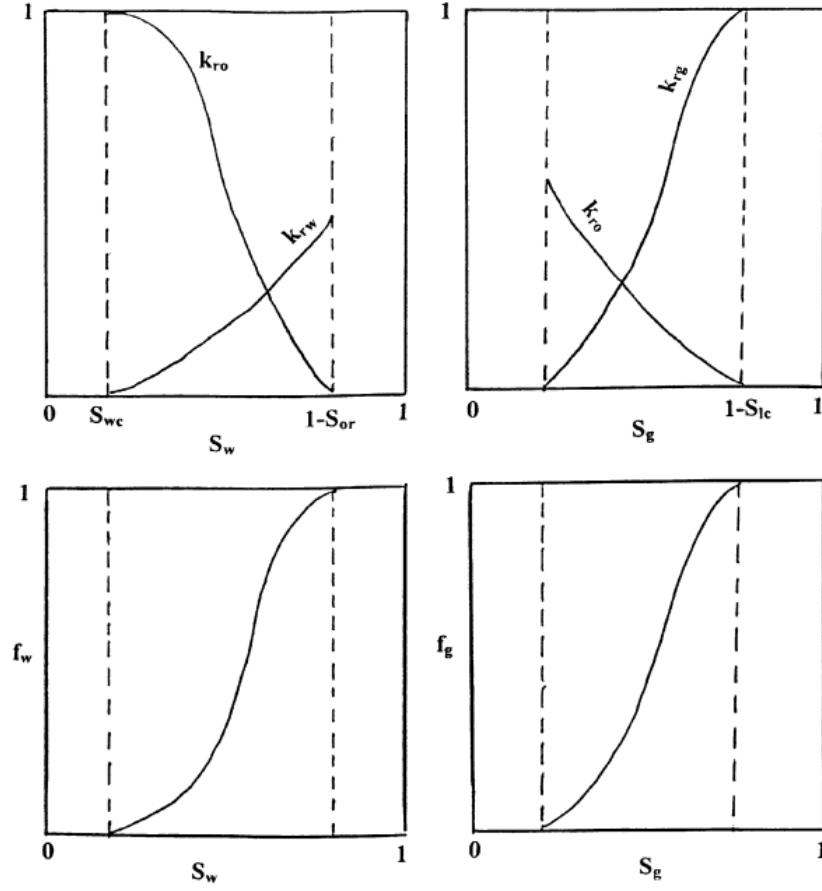


Figura 2.1: Curva de fluxo fracionário (f_w) característica, mostrando a sua forma de "S". Fonte: Adaptado de Ahmed, 2019 [1].

O desafio computacional reside em como obter os valores de $k_{ro}(S_w)$ e $k_{rw}(S_w)$ para qualquer valor de saturação de água S_w , uma vez que os dados de entrada podem vir de duas formas distintas: um modelo tabular ou um modelo analítico.

2.2.1 Implementação do Modelo Tabular (Interpolação Linear)

Quando os dados são fornecidos como uma tabela de pontos (lidos do arquivo de texto), o programa precisa calcular os valores de k_r para saturações que estão entre os pontos da tabela. Para isso, usaremos a Interpolação Linear.

- Algoritmo:

1. O software lerá o arquivo de entrada e armazenará os N pontos de dados em três vetores (ou listas) paralelos: `vec_Sw[]`, `vec_Krw[]`, `vec_Kro[]`.
2. Quando a função `calcularFw(Sw_desejada)` for chamada:
3. O programa irá buscar no `vec_Sw[]` os dois valores, $S_{w,i}$ e $S_{w,i+1}$, que "cerca" o valor de S_w desejado (ou seja, $S_{w,i} \leq S_w \leq S_{w,i+1}$).
4. A interpolação linear é aplicada para encontrar o k_{rw} e k_{ro} correspondentes:
 - $k_{rw} = k_{rw,i} + (S_{w,desejada} - S_{w,i}) \times \frac{k_{rw,i+1} - k_{rw,i}}{S_{w,i+1} - S_{w,i}}$
 - $k_{ro} = k_{ro,i} + (S_{w,desejada} - S_{w,i}) \times \frac{k_{ro,i+1} - k_{ro,i}}{S_{w,i+1} - S_{w,i}}$
5. Estes valores interpolados de k_{rw} e k_{ro} são então usados na equação principal de f_w .
6. Tratamento de Pontas: Se $S_{w,desejada}$ for menor que o primeiro valor da tabela, os valores do primeiro ponto ($k_{rw,0}, k_{ro,0}$) serão usados. Se for maior que o último ponto, os valores do último ponto ($k_{rw,N}, k_{ro,N}$) serão usados (sem extrapolação).

2.2.2 Implementação do Modelo Analítico (Correlações de Corey)

Quando o arquivo de entrada especifica o uso de um modelo analítico como o de Corey [3], o programa não usará uma tabela, mas sim um conjunto de parâmetros para calcular k_r diretamente.

- Algoritmo:
 1. O software lerá do arquivo de entrada os parâmetros do modelo de Corey:
 - S_{wir} : Saturação de água irreduzível.
 - S_{orw} : Saturação de óleo residual à injeção de água.
 - $k_{rw,max}$ (ou k_{rwro}): Permeabilidade relativa à água na saturação de óleo residual.
 - $k_{ro,max}$ (ou k_{rocw}): Permeabilidade relativa ao óleo na saturação de água irreduzível.
 - n_w : Expoente de Corey para a fase água.
 - n_o : Expoente de Corey para a fase óleo.
 2. Primeiro, calcula-se a saturação de água normalizada ($S_{w,norm}$) para qualquer S_w de entrada:
 - $S_{w,norm} = \frac{S_w - S_{wir}}{1 - S_{wir} - S_{orw}}$

3. A $S_{w,norm}$ é limitada entre 0 e 1 (se $S_w < S_{wir}$, $S_{w,norm} = 0$; se $S_w > (1 - S_{orw})$, $S_{w,norm} = 1$).
4. As permeabilidades relativas são então calculadas usando $S_{w,norm}$:
 - $k_{rw}(S_w) = k_{rw,max} \times (S_{w,norm})^{n_w}$
 - $k_{ro}(S_w) = k_{ro,max} \times (1 - S_{w,norm})^{n_o}$
5. Estes valores calculados de k_{rw} e k_{ro} são então usados na equação principal de f_w .

A estrutura do software em C++ será projetada de forma que a calculadora principal de f_w não precise saber qual desses métodos está sendo usado, apenas que ela pode pedir os valores de k_r .

Para uma dedução mais detalhada destas equações e uma discussão aprofundada sobre as propriedades dos fluidos e modelos de permeabilidade, consulte as referências clássicas da engenharia de reservatórios, como Ahmed [1] e Lake [4].

2.3 Identificação de pacotes – assuntos

Identificamos os seguintes pacotes (assuntos) lógicos para a organização do sistema:

- **Pacote Core (Núcleo):** Contém a classe principal com a lógica de engenharia, ou seja, a `CalculadoraFluxoFracionario`. Este pacote implementa a equação de Buckley-Leverett.
- **Pacote ModelosKr:** Contém a interface `ICurvasPermeabilidade` e suas implementações concretas (`CurvasPermeabilidadeTabelada` e `CurvasPermeabilidadeCorey`). Este pacote é responsável por fornecer os dados de permeabilidade relativa para o pacote Core.
- **Pacote IO (Entrada/Saída):** Contém classes utilitárias responsáveis pela leitura e *parsing* do arquivo de entrada de texto e pela escrita do arquivo de saída `.csv`.
- **Pacote Visualizacao:** Contém a classe utilitária `Gnuplot`, responsável por formatar os dados de saída e invocar o processo externo do `Gnuplot` para plotagem.
- **Pacote Aplicacao:** Contém a classe `Simulador` (com a função `main()`), que orquestra todo o processo: lê os dados usando `IO`, instancia o `ModelosKr` correto, instancia o `Core` e, por fim, chama a `Visualizacao`.

2.4 Diagrama de pacotes – assuntos

O Diagrama de Pacotes, apresentado na Figura 2.2, ilustra as dependências entre os assuntos lógicos identificados.

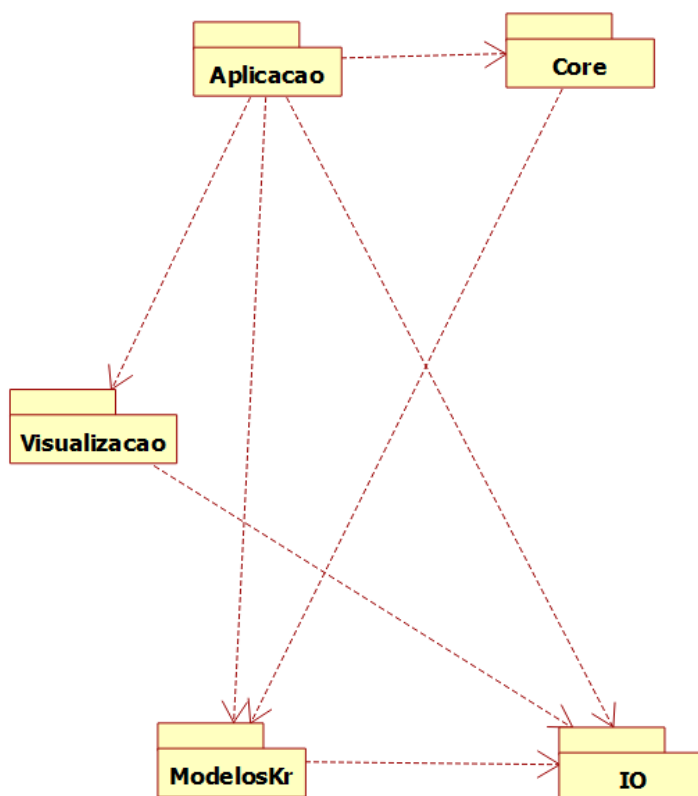


Figura 2.2: Diagrama de Pacotes.

Capítulo 3

AOO – Análise Orientada a Objeto

3.1 Diagramas de classes

O design estático do sistema, apresentado na Figura 3.1, é fundamentado em princípios S.O.L.I.D. de engenharia de software para garantir um código limpo, extensível e de fácil manutenção.

1. Princípio da Inversão de Dependência (D do S.O.L.I.D.): Este é o pilar central da arquitetura. O módulo de alto nível (`CalculadoraFluxoFracionario`) não depende diretamente dos módulos de baixo nível (`...Tabelada`, `...Corey`). Ambos dependem de uma abstração: a interface `ICurvasPermeabilidade`.
2. Padrão de Projeto *Strategy* (Estratégia): A interface `ICurvasPermeabilidade` implementa o padrão *Strategy*. Ela define um "contrato" para uma família de algoritmos (os modelos de k_r) e permite que eles sejam trocados dinamicamente. A Calculadora é configurada com uma dessas estratégias no momento da sua criação, através de Injeção de Dependência em seu construtor. Isso torna a calculadora completamente agnóstica aos detalhes de como o k_r é obtido.
3. Padrão de Projeto Facade (Fachada): A classe `Gnuplot` é uma Facade estática. Ela fornece uma interface unificada e simples (`plotarCurva()`) para um subsistema complexo (formatação de *scripts*, salvamento de arquivos temporários e chamada de processos do sistema operacional).

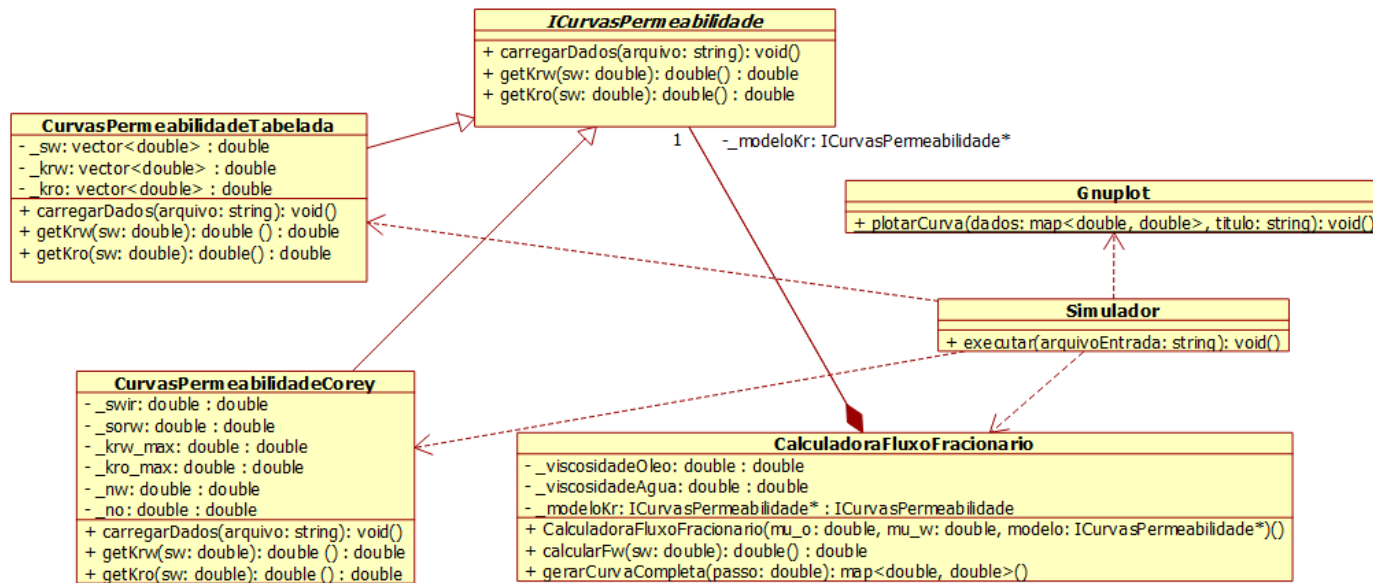


Figura 3.1: Diagrama de classes.

3.1.1 Dicionário de classes

- Classe **ICurvasPermeabilidade**: (Interface) Define o contrato de serviço para os modelos de permeabilidade relativa. Garante o polimorfismo e a Inversão de Dependência.
- Classe **CurvasPermeabilidadeTabelada**: (Implementação Concreta) Implementação da *Strategy* de k_r via dados tabulares. Encapsula a lógica de interpolação linear.
- Classe **CurvasPermeabilidadeCorey**: (Implementação Concreta) Implementação da *Strategy* de k_r via correlação analítica de Corey [3]. Encapsula a lógica matemática do modelo.
- Classe **CalculadoraFluxoFracionario**: (Domínio de Negócio) Classe principal que implementa o domínio central de negócio (a equação de Buckley-Leverett). Opera em um alto nível de abstração, dependendo apenas da interface **ICurvasPermeabilidade**.
- Classe **Gnuplot**: (Utilitário / Facade) Classe estática que encapsula toda a lógica de interação com o Gnuplot, simplificando a plotagem para o resto da aplicação.
- Classe **Simulador**: (Orquestrador / Ponto de Entrada) Atua como o orquestrador da aplicação. É responsável por "montar" o sistema (ler arquivos, instanciar os objetos corretos usando *new*) e injetar as dependências (*modeloKr* dentro da *Calculadora*). Contém o ponto de entrada `main()`.

3.2 Diagrama de sequência – eventos e mensagens

3.2.1 Diagrama de sequência geral

O diagrama de sequência geral, Figura 3.2, ilustra o fluxo principal de execução do software, correspondente ao caso de uso "Executar Cálculo e Analisar Resultados" com sucesso.

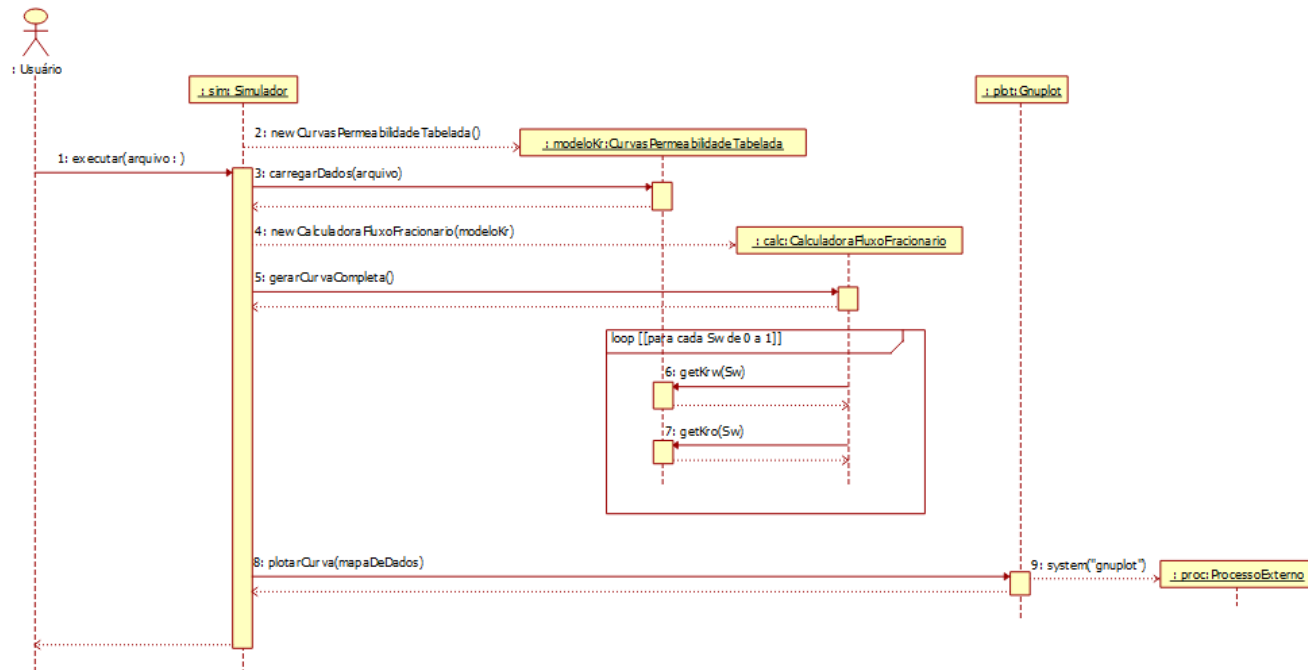


Figura 3.2: Diagrama de sequência.

3.2.2 Diagrama de sequência específico

O diagrama da Figura 3.3 detalha a atividade específica de `getKrw()` na classe `CurvasPermeabilidadeTabelada`, mostrando a lógica de interpolação.

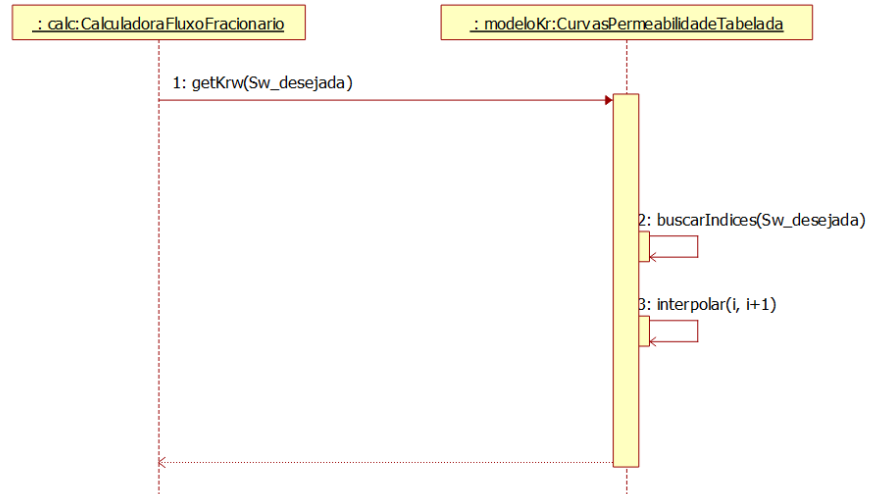


Figura 3.3: Diagrama de sequência.

3.3 Diagrama de comunicação – colaboração

O diagrama de comunicação da Figura 3.3 apresenta a mesma interação do diagrama de sequência geral, mas foca na colaboração e nas mensagens trocadas entre os objetos, em vez da ordem temporal.

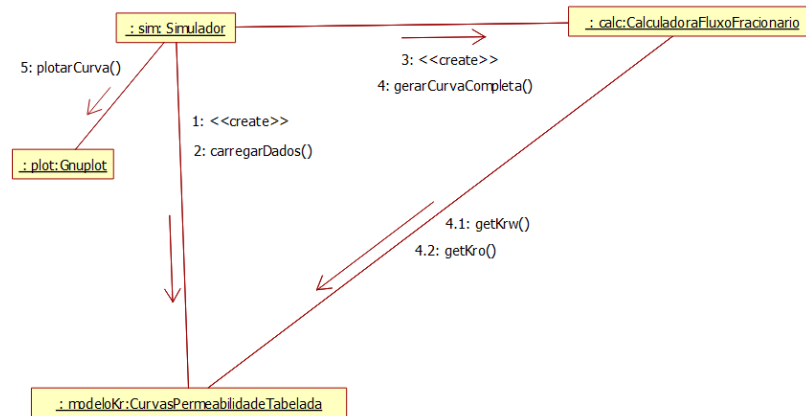


Figura 3.4: Diagrama de comunicação.

3.4 Diagrama de estado

O diagrama da Figura 3.5 modela os estados dinâmicos de um objeto *Simulador* ao longo de sua vida, desde a criação até a finalização da execução.

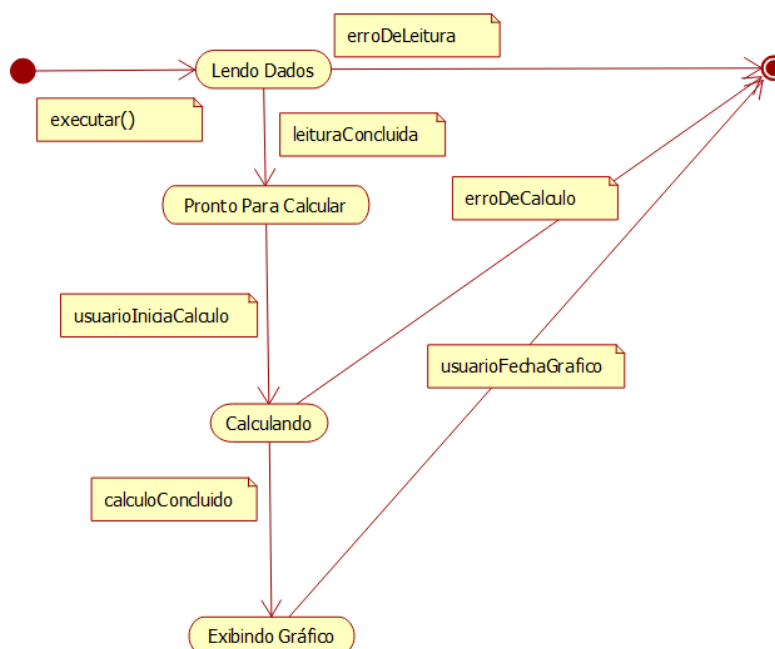


Figura 3.5: Diagrama de máquina de estado.

3.5 Diagrama de atividades

O diagrama de atividades da Figura 3.6 detalha o algoritmo do método `getKrw()` na classe `CurvasPermeabilidadeTabelada`, como descrito na Formulação Teórica. Este é um cálculo específico e crucial.

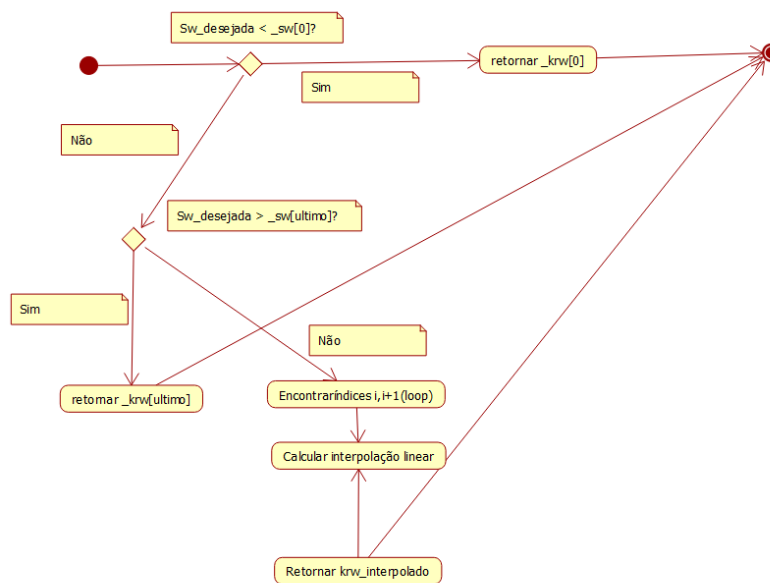


Figura 3.6: Diagrama de atividades.

Capítulo 4

Projeto

4.1 Projeto do sistema

1. Protocolos

- Comunicação Externa: O protocolo de comunicação com o usuário será via arquivos de texto. O formato do arquivo de entrada (detalhado no Apêndice A) é o protocolo principal. O sistema gerará um arquivo `.csv` (valores separados por vírgula) como protocolo de saída, garantindo fácil interoperabilidade.
- Comunicação Interna: A comunicação entre os objetos do sistema seguirá um padrão de passagem de mensagens síncronas (chamadas de método diretas).
- API: A API interna da biblioteca de cálculo será definida pela interface (classe base abstrata) `ICurvasPermeabilidade` e pela classe `CalculadoraFluxoFracionario`.

2. Recursos

- Banco de Dados: Não será utilizado um banco de dados. Para o escopo da aplicação (uma ferramenta de análise leve), um BD introduziria uma complexidade de implantação e configuração desnecessária.
- Armazenamento: A persistência dos dados de entrada é gerenciada pelo usuário através de arquivos `.txt`.

3. Controle

- Implementação: O controle do sistema será sequencial e baseado em procedimentos. A execução seguirá um fluxo linear (Ler -> Calcular -> Plotar).

- Concorrência: Não há necessidade de concorrência ou processamento paralelo (*multithreading*), pois o custo computacional do cálculo do fluxo fracionário 1D é baixo e a execução é instantânea.

4. Plataformas

- Hardware/SO: O software será desenvolvido em C++ padrão (C++17), garantindo alta portabilidade e compilação nativa em *Windows* e GNU/Linux.
- Bibliotecas Externas: A única dependência externa será o Gnuplot. Para manter o software leve, o Gnuplot não será linkado ao código; ele será invocado como um processo de sistema externo (`system()`).
- IDE: O desenvolvimento será feito utilizando um editor de texto (Kate) e compilação via *make* no terminal MSYS2.

5. Padrões de projeto

- Conforme detalhado no Capítulo 3, o design da arquitetura é centrado no uso dos padrões de projeto *Strategy* (para os modelos de permeabilidade) e *Facade* (para a classe Gnuplot).

4.2 Projeto orientado a objeto – POO

4.2.1 Efeitos do projeto no modelo estrutural:

- A arquitetura definida na Análise (Cap. 3) é mantida. O uso do padrão *Strategy* garante que o Diagrama de Pacotes seja limpo: o pacote *Core* depende apenas da abstração em *ModelosKr*, não das implementações concretas.

4.2.2 Efeitos do projeto no modelo dinâmico:

- Os diagramas de sequência e estado confirmam o fluxo de controle síncrono e sequencial. A chamada ao Gnuplot (`system()`) será bloqueante, garantindo que o programa só termine após o usuário fechar o gráfico.

4.2.3 Efeitos do projeto nos atributos:

- Todos os atributos de classe serão privados (*private*), garantindo encapsulamento. A

`CalculadoraFluxoFracionario` armazenará um ponteiro para a interface `ICurvasPermeabilidade`, minimizando o acoplamento.

4.2.4 Efeitos do projeto nos métodos:

- Os métodos de cálculo (como `getKrw`, `calcularFw`) serão declarados como `const` sempre que possível, para garantir que não modifiquem o estado do objeto. O método `plotarCurva` da classe `Gnuplot` será `static`, pois não depende de um estado de instância.

4.2.5 Efeitos do projeto nas heranças:

- A herança será usada exclusivamente para implementar interfaces (herança de tipo), não para reutilização de código. Isso segue o princípio de "Composição sobre Herança".

4.2.6 Efeitos do projeto nas associações:

- A associação principal é a de uso (agregação), onde a Calculadora usa a interface `ICurvasPermeabilidade` que lhe é injetada. O Simulador gerencia o ciclo de vida (criação `new` e destruição `delete`) dos objetos.

4.2.7 Efeitos do projeto nas otimizações:

- Nenhuma otimização de baixo nível (ex: paralelismo) é necessária. A otimização do projeto está no design de O.O., que evita recálculos e promove baixo acoplamento.

4.3 Diagrama de componentes

O Diagrama de Componentes (Figura 4.1) detalha as dependências de compilação e ligação do software. O componente executável final, `fw_calc`, depende da compilação e linkagem de múltiplos componentes de código-fonte (`.cpp`). Cada componente `.cpp` (ex: `Simulador.cpp`, `Calculadora.cpp`) depende do seu respectivo arquivo de cabeçalho (`.h`) e das abstrações das quais depende (ex: `Calculadora.cpp` depende de `ICurvasPermeabilidade.h`). O componente `Gnuplot.cpp` não depende do executável do Gnuplot em tempo de compilação, mas o executável `fw_calc` tem uma dependência em tempo de execução com o Gnuplot.

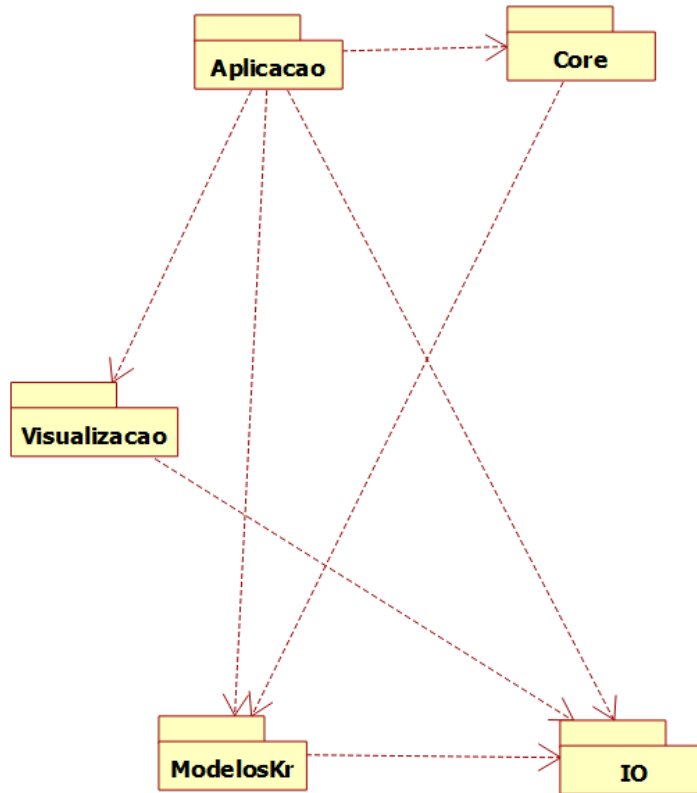


Figura 4.1: Diagrama de componentes.

4.4 Diagrama de implantação

O Diagrama de Implantação (Figura 4.2) para este projeto é simples, pois não se trata de um sistema distribuído. Haverá um único nó principal: `<<device>>` Computador Pessoal. Este nó de hardware hospeda o sistema operacional (ex: *Windows* 10/11 ou GNU/Linux). Dentro deste S.O., residirão os dois artefatos de software necessários para a execução:

1. `[fw_calc]` : O nosso artefato executável compilado.
2. `[gnuplot]`: O artefato executável da dependência externa. O `[fw_calc]` possui uma dependência de execução (do tipo `<<call>>`) com o `[gnuplot]`.

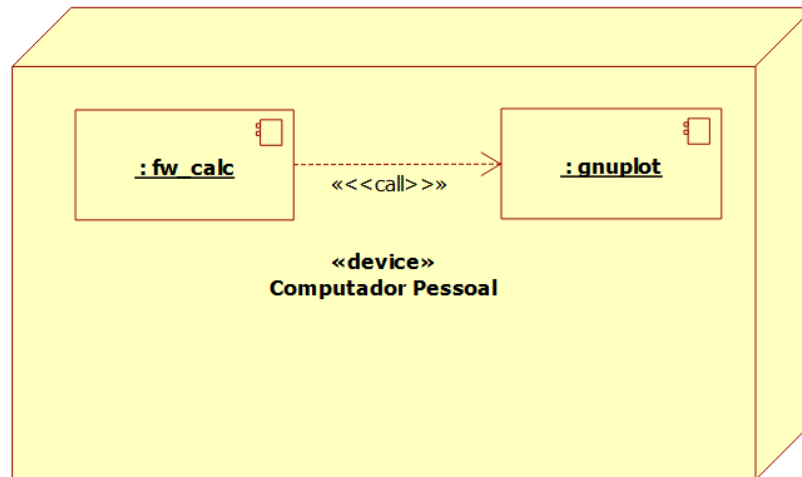


Figura 4.2: Diagrama de implantação.

Nota:

Não perca de vista a visão do todo; do projeto de engenharia como um todo. Cada capítulo, cada seção, cada parágrafo deve se encaixar. Este é um diferencial fundamental do engenheiro em relação ao técnico, a capacidade de desenvolver projetos, de ver o todo e suas diferentes partes, de modelar processos/sistemas/produtos de engenharia.

4.4.1 Lista de características <<features>>

No final do ciclo de concepção e análise chegamos a uma lista de características <<features>> que teremos de implementar.

Após a análises desenvolvidas e considerando o requisito de que este material deve ter um formato didático, chegamos a seguinte lista:

- v0.1
 - Implementar classes Simulador, CalculadoraFluxoFracionario.
 - Implementar ICurvasPermeabilidade e CurvasPermeabilidadeTabelada.
 - Implementar leitor de arquivo de entrada (parser) para o modelo tabelado e viscosidades.
 - Implementar cálculo da curva e salvamento em arquivo output.csv.
 - Testes: Validar o .csv de saída contra um cálculo manual em planilha.

- v0.2
 - Implementar classe `CurvasPermeabilidadeCorey` [3].
 - Atualizar o parser do arquivo de entrada para ler os parâmetros do modelo Corey .
 - Testes: Validar o `.csv` de saída para o modelo Corey.
- v0.3
 - Implementar classe `Gnuplot` .
 - Integrar a chamada do Gnuplot no Simulador para plotagem automática.
 - Testes: Validar a exibição do gráfico em *Windows* e *Linux*.

4.4.2 Tabela classificação sistema

A Tabela 4.1 a seguir é utilizada para classificação do sistema desenvolvido.

Tabela 4.1: Tabela classificação sistema.

Licença:	☒ livre GPL-v3 ☐ proprietária
Engenharia de software:	☐ tradicional ☒ ágil ☐ outras
Paradigma de programação:	☐ estruturada ☒ orientado a objeto - POO ☐ funcional
Modelagem UML:	☒ básica ☒ intermediária ☐ avançada
Algoritmos:	☒ alto nível ☐ baixo nível
	implementação: ☐ recursivo ou ☒ iterativo; ☒ determinístico ou ☐ não-determinístico; ☐ exato ou ☒ aproximado
	concorrências: ☒ serial ☐ concorrente ☐ paralelo
	paradigma: ☒ dividir para conquistar ☐ programação linear ☐ transformação/ redução ☐ busca e enumeração ☐ heurístico e probabilístico ☐ baseados em pilhas
Software:	☐ de base ☒ aplicados ☐ de cunho geral ☒ específicos para determinada área ☒ educativo ☒ científico
	instruções: ☒ alto nível ☐ baixo nível
	otimização: ☒ serial não otimizado ☐ serial otimizado ☐ concorrente ☐ paralelo ☐ vetorial
	interface do usuário: ☐ kernel numérico ☒ linha de comando ☐ modo texto ☒ híbrida (texto e saídas gráficas) ☐ modo gráfico (ex: Qt) ☐ navegador
Recursos de C++:	☒ C++ básico (FCC): variáveis padrões da linguagem, estruturas de controle e repetição, estruturas de dados, struct, classes(objetos, atributos, métodos), funções; entrada e saída de dados (<i>streams</i>), funções de cmath
	☒ C++ intermediário: funções lambda. Ponteiros, referências, herança, herança múltipla, polimorfismo, sobrecarga de funções e de operadores, tipos genéricos (templates), <i>smart pointers</i> . Diretrizes de pré-processador, classes de armazenamento e modificadores de acesso. Estruturas de dados: enum, uniões. Bibliotecas: entrada e saída acesso com arquivos de disco, redirecionamento. Bibliotecas: <i>filesystem</i>
	☒ C++ intermediário 2: A biblioteca de gabaritos de C++ (a STL), containers, iteradores, objetos funções e funções genéricas. Noções de processamento paralelo (múltiplas threads, uso de <i>thread</i> , <i>join</i> e <i>mutex</i>). Bibliotecas: <i>random</i> , <i>threads</i>
	☐ C++ avançado: Conversão de tipos do usuário, especializações de templates, excessões. Cluster de computadores, processamento paralelo e concorrente, múltiplos processos (pipes, memória compartilhada, sinais). Bibliotecas: <i>expressões regulares</i> , <i>múltiplos processos</i>
Bibliotecas de C++:	☒ Entrada e saída de dados (<i>streams</i>) ☒ cmath ☒ <i>filesystem</i> ☐ <i>random</i> ☐ <i>threads</i> ☐ <i>expressões regulares</i> ☐ <i>múltiplos processos</i>
Bibliotecas externas:	☒ CGnuplot ☐ QCustomPlot ☐ Qt diálogos ☐ QT Janelas/menus/BT
Ferramentas auxiliares:	Montador: ☒ make ☐ cmake ☐ qmake
IDE:	☒ Editor simples: kate/gedit/emacs ☐ kdevelop ☐ QT-Creator ☐
SCV:	☐ cvs ☐ svn ☒ git
Disciplinas correlacionadas	☐ estatística ☒ cálculo numérico ☐ modelamento numérico ☐ análise e processamento de imagens

Capítulo 5

Teste

Todo projeto de engenharia passa por uma etapa de testes. Neste capítulo apresentamos alguns testes do software desenvolvido. Estes testes devem dar resposta aos diagramas de caso de uso inicialmente apresentados (diagramas de caso de uso geral e específicos).

5.1 Teste 1: Validando o Ciclo 1 - Modelo Tabe-

lado

Descrição do Teste:

- O que está sendo testado? Este teste valida o funcionamento básico do núcleo numérico e o algoritmo de interpolação linear. O objetivo é garantir que o software consiga ler uma tabela de permeabilidade relativa genérica e calcular valores intermediários corretamente.
- Como será realizado? O software é executado com o arquivo de entrada `test/Teste-01.in`, que contém uma tabela simplificada com 7 pontos de k_{rw} e k_{ro} .
- Como será validado? A validação é feita pela análise da curva gerada no arquivo `output.csv`. Verifica-se se a curva é contínua e se os pontos calculados correspondem a uma interpolação linear suave entre os pontos de entrada, sem saltos ou descontinuidades numéricas.

5.1.1 Arquivo de entrada de dados

Apresenta-se a seguir o arquivo de entrada `Teste-01.in`, Listagem 5.1, que utiliza o modelo tabelado.

Listagem 5.1: `Teste-01.in`.

```
# Exemplo de arquivo de entrada para modelo tabelado
```



```
VISC_OLEO 1.5
VISC_AGUA 0.8
MODELO_KR TABELADO #SW/KRW/KRO
DADOS_KR_INICIO
0.20 0.00 0.90
0.30 0.05 0.75
0.40 0.12 0.50
0.50 0.20 0.30
0.60 0.30 0.15
0.70 0.40 0.05
0.80 0.50 0.00
FIM_DADOS
```

5.1.2 Arquivo de saída de dados (resultados da tela)

O software é de linha de comando e não gera saída na tela, exceto por mensagens de status ou erro. O resultado principal é o arquivo `output.csv`, que conterà duas colunas: S_w e f_w .

5.1.3 Figuras

A Figura 5.1 mostra a execução do programa no terminal. A Figura 5.2 mostra o gráfico gerado a partir do `output.csv`, comprovando a geração da curva "S" característica.

A Figura 5.2 apresenta o gráfico gerado. A curva suavizada confirma que a interpolação linear entre os pontos fornecidos está funcionando corretamente, sem descontinuidades.

```

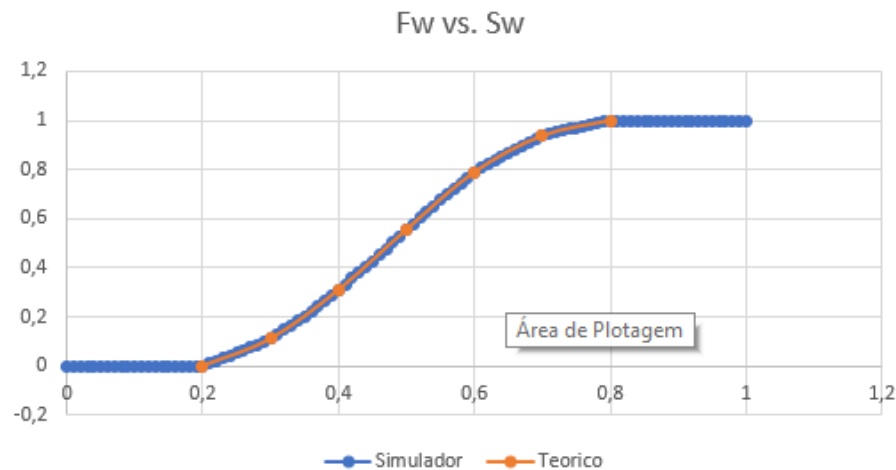
C:\Users\Fabia\OneDrive\Documentos\Disciplinas UENF 2025.2\Disciplina de Projeto\projeto-git\LDSC-ProjetEngenharia-2-Software-TituloProjeto-ModeloCompleto\4-Execucao-Detalhamento-Construcao-Versao-1
$ cd "/C:/Users/Fabia/OneDrive/Documentos/Disciplinas UENF 2025.2/Disciplina de Projeto/projeto-git/LDSC-ProjetEngenharia-2-Software-TituloProjeto-ModeloCompleto/4-Execucao-Detalhamento-Construcao-Versao-1/"
$ cd "/C:/Users/Fabia/OneDrive/Documentos/Disciplinas UENF 2025.2/Disciplina de Projeto/projeto-git/LDSC-ProjetEngenharia-2-Software-TituloProjeto-ModeloCompleto/4-Execucao-Detalhamento-Construcao-Versao-1/"
$ make clean
Arquivos de compilação limpos.

C:\Users\Fabia\OneDrive\Documentos\Disciplinas UENF 2025.2\Disciplina de Projeto\projeto-git\LDSC-ProjetEngenharia-2-Software-TituloProjeto-ModeloCompleto\4-Execucao-Detalhamento-Construcao-Versao-1
$ make
g++ -std=c++17 -Wall -g -Isrc -c src/main.cpp -o obj/main.o
g++ -std=c++17 -Wall -g -Isrc -c src/Simulador.cpp -o obj/Simulador.o
g++ -std=c++17 -Wall -g -Isrc -c src/CalculadoraFluxoFracionario.cpp -o obj/CalculadoraFluxoFracionario.o
g++ -std=c++17 -Wall -g -Isrc -c src/CurvasPermeabilidadeTabelada.cpp -o obj/CurvasPermeabilidadeTabelada.o
g++ -std=c++17 -Wall -g -Isrc -c src/CurvasPermeabilidadeCorey.cpp -o obj/CurvasPermeabilidadeCorey.o
g++ -std=c++17 -Wall -g -Isrc -c src/Gnuplot.cpp -o obj/Gnuplot.o
g++ -std=c++17 -Wall -g -Isrc -o bin/fw_calc obj/main.o obj/Simulador.o obj/CalculadoraFluxoFracionario.o obj/CurvasPermeabilidadeTabelada.o obj/CurvasPermeabilidadeCorey.o obj/Gnuplot.o
Compilação concluída! Executável está em bin/fw_calc

C:\Users\Fabia\OneDrive\Documentos\Disciplinas UENF 2025.2\Disciplina de Projeto\projeto-git\LDSC-ProjetEngenharia-2-Software-TituloProjeto-ModeloCompleto\4-Execucao-Detalhamento-Construcao-Versao-1
$ ./bin/fw_calc test/Teste-01.in
Iniciando simulador...
Lendo arquivo de configuracao: test/Teste-01.in
Modelo selecionado: TABELADO
DEBUG: 7 pontos de Kr tabelados foram carregados.
Calculando curvas...
Plotando resultados...
DEBUG: Chamando Gnuplot...
Pressione Enter para fechar

```

Figura 5.1: Tela do programa mostrando terminal executando.

Figura 5.2: Tela do programa mostrando gráfico (*Excel*) gerado a partir do output.csv.

5.2 Teste 2: Validando o Ciclo 2 - Modelo Corey

Descrição do Teste:

- O que está sendo testado? Este teste valida a arquitetura de software (Padrão *Strategy*) e a implementação matemática do modelo analítico de Corey. O objetivo é provar que o sistema consegue trocar de modelo de cálculo dinamicamente apenas alterando o arquivo de entrada.
- Como será realizado? O software é executado com o arquivo `test/Teste-02.in`, que utiliza a palavra-chave `MODELO_KR COREY` e define os expoentes $n_w = 2.0$ e $n_o = 2.5$.
- Como será validado? A validação é visual e analítica. A curva resultante deve apresentar o perfil exponencial característico das equações de Corey, diferindo das linhas retas do modelo tabelado. Além disso, os pontos finais (saturação irreduzível de água e óleo residual) devem coincidir exatamente com os parâmetros S_{wir} e S_{orw} fornecidos no arquivo.

5.2.1 Arquivo de entrada de dados

Apresenta-se a seguir o arquivo de entrada `Teste-02.in`, Listagem 5.2.

Listagem 5.2: `Teste-02.in`.

```
# Exemplo de arquivo de entrada para modelo Corey
VISC_OLEO 2.0
VISC_AGUA 1.0
MODELO_KR COREY
COREY_SWIR      0.15
COREY_SORW      0.20
COREY_KRW_MAX   0.5
COREY_KRO_MAX   0.9
COREY_NW        2.0
COREY_NO        2.5
```

5.2.2 Arquivo de saída de dados (resultados da tela)

Similar ao Teste 1, o resultado principal é o arquivo `output.csv`.

5.2.3 Figuras

A Figura 5.3 mostra o resultado gráfico. A curva apresenta o comportamento exponencial característico do modelo de Corey, confirmando a correta implementação das equações de potência.

A curva gerada apresenta a concavidade característica das funções de potência, confirmando que os expoentes $n_w = 2.0$ e $n_o = 2.5$ foram aplicados corretamente na fórmula analítica.

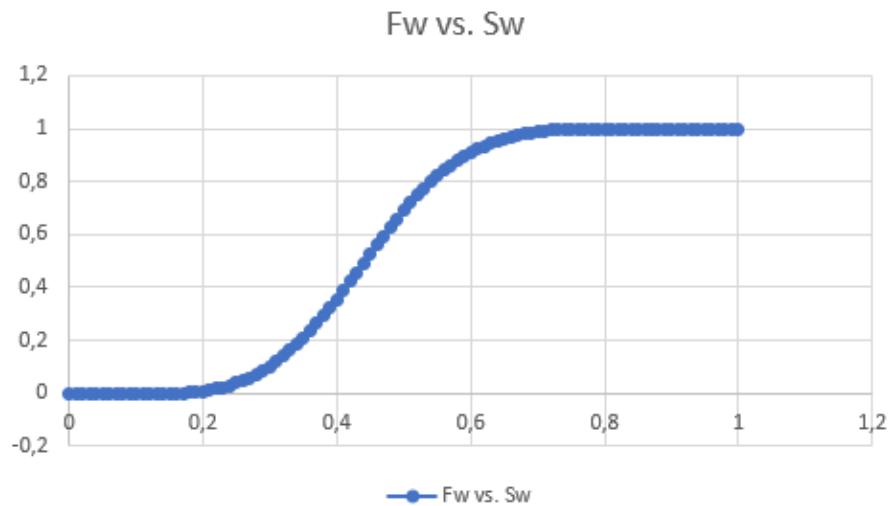


Figura 5.3: Tela do programa mostrando curva do modelo de Corey.

5.3 Teste 3: Validação com Literatura (Tarek Ahmed)

Descrição do Teste

- O que está sendo testado? Este é o teste de validação científica do projeto. O objetivo é comprovar a precisão física dos resultados comparando-os com um problema clássico da literatura de engenharia de reservatórios.
- Como será realizado? O software é executado com o arquivo `test/Teste-Tarek-1cp.in`. Este arquivo reproduz os parâmetros exatos do Exemplo 14-5 do livro *Reservoir Engineering Handbook* de Tarek Ahmed [1], considerando a viscosidade do óleo de 1.0 cp.
- Como será validado?
 1. Numérica: Os valores do arquivo de saída `output.csv` são comparados com a coluna " $\mu_o = 1.0$ " da tabela de resultados na página 950 do livro 1. O erro absoluto deve ser desprezível.
 2. Gráfica: A curva gerada (Figura 5.4) é comparada visualmente com a Figura 5.5 do livro, buscando a reprodução fiel da curva "S".

5.3.1 Arquivo de entrada de dados

Apresenta-se a seguir o conteúdo do arquivo `Teste-Tarek-1cp.in` utilizado para esta simulação, Listagem 5.3.

Listagem 5.3: Teste-Tarek-1cp.in.

```
# Teste de Validação – Exemplo 14–5 do Tarek Ahmed
# Viscosidade do Oleo = 1.0 cp
# Viscosidade da Agua = 0.5 cp

VISC_OLEO 1.0
VISC_AGUA 0.5

MODELO_KR TABELADO

# Dados lidos da Figura 14–15 (Tabela pg. 950)
DADOS_KR_INICIO
0.24 0.00 0.95
0.30 0.01 0.89
0.40 0.04 0.74
0.50 0.09 0.45
0.60 0.17 0.19
0.65 0.28 0.12
0.70 0.22 0.06
0.75 0.36 0.03
0.78 0.41 0.00
FIM_DADOS
```

A Figura 5.4 apresenta o gráfico gerado automaticamente pelo software após a execução do teste.

A análise comparativa com a Figura 5.5 demonstra que a curva obtida reproduz com exatidão o comportamento esperado descrito por Ahmed [1]. Os pontos de saturação inicial de água ($S_{wi} \approx 0.24$) e saturação de óleo residual ($S_{or} \approx 0.22$, ou seja, $S_w \approx 0.78$) coincidem perfeitamente com os limites da tabela de entrada, e a curvatura "S" reflete corretamente a variação da razão de mobilidade ao longo do domínio de saturação. Isso confirma que o núcleo numérico da ferramenta é robusto e confiável para aplicações de engenharia.

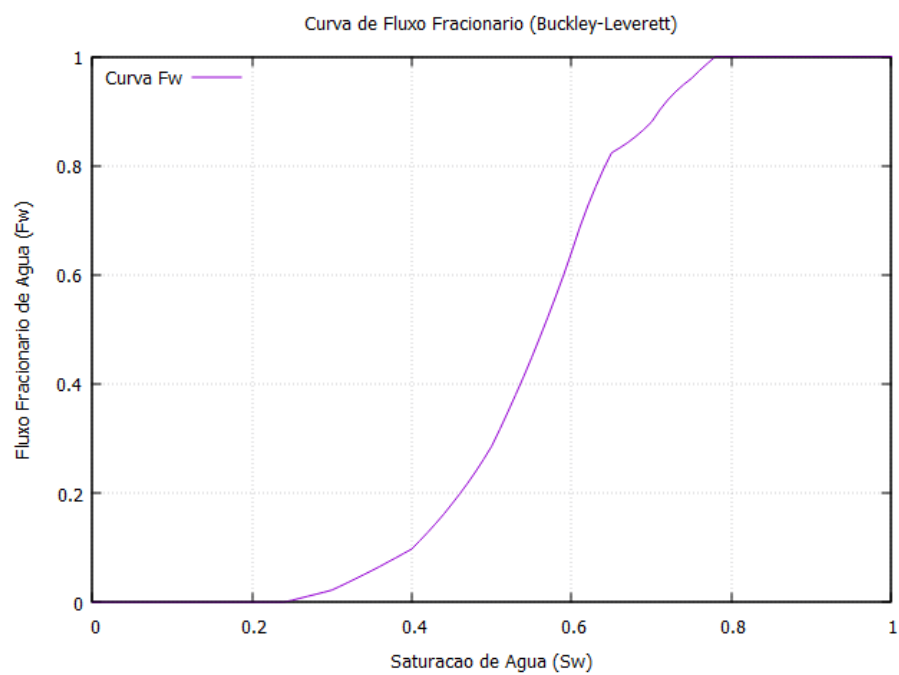


Figura 5.4: Tela do programa mostrando curva do Exemplo 14-5 do livro *Reservoir Engineering Handbook* de Tarek Ahmed [1] (com $\mu_o = 1.0$).

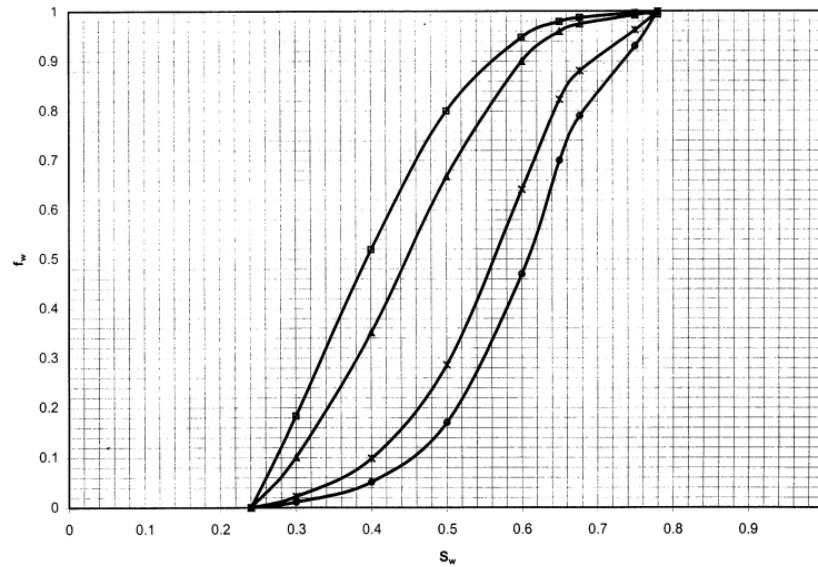


Figure 14-16. Effect of μ_o on f_w .

Figura 5.5: Curva do Exemplo 14-5 do livro *Reservoir Engineering Handbook* de Tarek Ahmed [1] (com $\mu_o = 0.5, \mu_o = 1.0, \mu_o = 5$ e $\mu_o = 10$).

Observar que, a Figura 5.4 representa a curva com viscosidade do óleo igual a 10 ($\mu_o = 10$), para comparação com a devida curva da imagem 5.5.

5.4 Teste 4: Validação da Camada de Apresentação (Gnuplot)

Descrição do Teste:

- O que está sendo testado? Este teste valida o requisito não funcional de usabilidade e a integração com a ferramenta externa Gnuplot (Padrão Facade).
- Como será realizado? O teste consiste na execução bem-sucedida de qualquer um dos cenários anteriores em um ambiente *Desktop* (*Windows* ou *Linux*).
- Como será validado? O teste é considerado aprovado se, após o cálculo, uma janela gráfica do Gnuplot for aberta automaticamente, exibindo a curva correta com títulos, eixos nomeados e grade (*grid*) ativada, sem a

necessidade de intervenção manual do usuário para plotar os dados. A persistência da janela (ela não deve fechar sozinha imediatamente) também é um critério de sucesso.

5.4.1 Arquivo de entrada de dados

Este teste reutiliza o arquivo `Teste-01.in`. Além da saída padrão (`output.csv`), o resultado esperado é uma ação do sistema: a abertura de uma janela de gráfico.

5.4.2 Figuras

A Figura 5.6 mostra uma captura de tela a janela do Gnuplot que foi aberta automaticamente pelo software, exibindo o resultado gráfico.

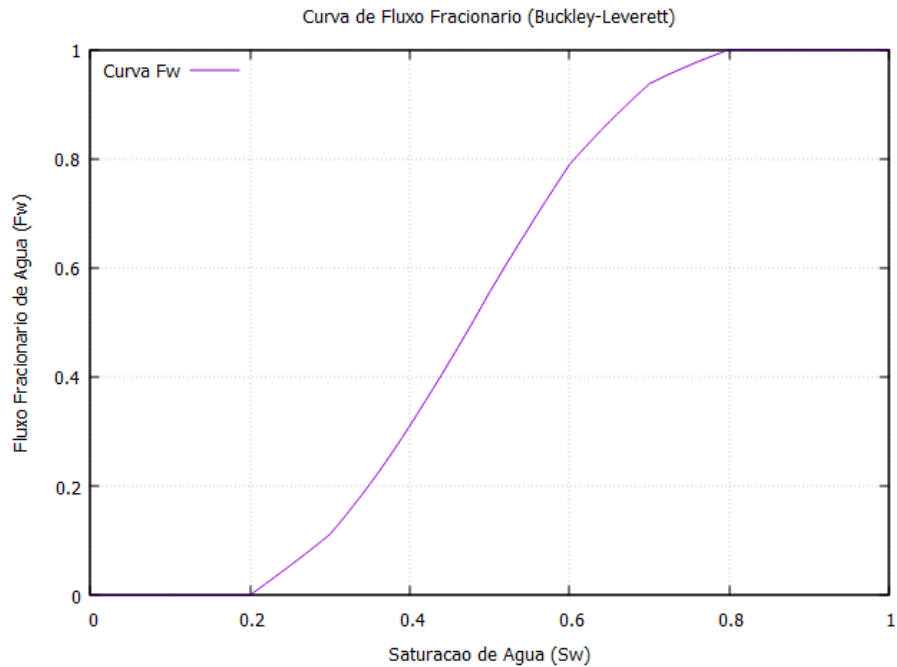


Figura 5.6: Tela do programa mostrando captura de tela a janela do Gnuplot.

Referências Bibliográficas

- [1] Tarek Ahmed. *Reservoir Engineering Handbook*. Gulf Publishing Company, 4th edition, 2010.
- [2] S. E. Buckley and M. C. Leverett. Mechanism of Fluid Displacement in Sands. *Transactions of the AIME*, 146:107–116, 1942.
- [3] A. T. Corey. The Interrelation Between Gas and Oil Relative Permeabilities. *Producers Monthly*, 19(1):38–41, 1954.
- [4] Larry W. Lake et al. *Fundamentals of Enhanced Oil Recovery*. Society of Petroleum Engineers, 2010.

Apêndice A

Formato do Arquivo de Entrada

O software utiliza um arquivo de entrada de texto `.txt` baseado em palavras-chave. O parser identifica a palavra-chave (ex: `VISC_OLEO`) e lê o valor na mesma linha.

A palavra-chave `MODELO_KR` define qual modelo de permeabilidade será usado: `TABELADO` ou `COREY` .

A.1 Exemplo de Arquivo de Entrada (Modelo Tabelado)

A seguir a Listagem A.1, com um exemplo de arquivo de entrada para modelo tabelado.

Listagem A.1: Exemplo de arquivo de entrada para modelo tabelado.

```
# Exemplo de arquivo de entrada para modelo tabelado
# Comentários são ignorados

VISC_OLEO 1.5
VISC_AGUA 0.8

MODELO_KR TABELADO

# Definição da tabela Sw, Krw, Kro
# Os dados são lidos até a palavra-chave FIM_DADOS
DADOS_KR_INICIO
0.20 0.00 0.90
0.30 0.05 0.75
0.40 0.12 0.50
```

```
0.50 0.20 0.30
0.60 0.30 0.15
0.70 0.40 0.05
0.80 0.50 0.00
FIM_DADOS
```

A.2 Exemplo de Arquivo de Entrada (Modelo Corey)

A seguir a Listagem A.2, com um exemplo de arquivo de entrada para modelo Corey.

Listagem A.2: Exemplo de arquivo de entrada para modelo Corey.

```
# Exemplo de arquivo de entrada para modelo Corey
# Comentários são ignorados

VISC_OLEO 2.0
VISC_AGUA 1.0

MODELO_KR COREY

# Parâmetros da correlação de Corey
COREY_SWIR      0.15
COREY_SORW      0.20
COREY_KRW_MAX   0.5
COREY_KRO_MAX   0.9
COREY_NW        2.0
COREY_NO        2.5
```

Apêndice B

Guia de Compilação e Execução

Este apêndice descreve os passos exatos para compilar e executar o software "Ferramenta de Fluxo Fracionário" em um ambiente *Windows*, utilizando o terminal MSYS2 MINGW64.

B.1 Pré-requisitos

Certifique-se de que o ambiente MSYS2 está instalado com os pacotes `mingw-w64-x86_64-toolchain` (que inclui `g++` e `make`) e `mingw-w64-x86_64-gnuplot`.

B.2 Compilação do Projeto

1. Abra o terminal MSYS2 MINGW64.
2. Navegue até a pasta raiz do projeto (onde está o arquivo `Makefile`). Exemplo, Listagem B.1:

Listagem B.1: Comando para navegar até a pasta do projeto.

```
cd "/c/Users/SeuUsuario/Documents/ProjetoFluxoFracionario/"
```

3. Execute o comando de limpeza para garantir uma compilação limpa, Listagem B.2:

Listagem B.2: Limpando arquivos antigos.

```
make clean
```

4. Compile o projeto usando o comando `make`. O `Makefile` irá gerenciar a compilação de todos os arquivos `.cpp` e gerar o executável na pasta `bin/`, Listagem B.3.

Listagem B.3: Comando de compilação.

```
make
```

B.3 Execução e Teste

Após a compilação bem-sucedida (mensagem "Compilação concluída!"), o programa pode ser executado.

1. Para rodar o Teste 1 (Modelo Tabelado), execute, Listagem B.4:

Listagem B.4: Executando o Teste 1.

```
./bin/fw_calc test/Teste-01.in
```

2. Para rodar o Teste de Validação (Dados de Tarek Ahmed,[1]), execute, Listagem B.5:

Listagem B.5: Executando a Validação com Literatura.

```
./bin/fw_calc test/Teste-Tarek-1cp.in
```

Se o ambiente estiver configurado corretamente, uma janela do Gnuplot abrirá automaticamente exibindo o gráfico da curva de fluxo fracionário.